Section: CS July 7, 2026

Multi-Party Authorization Proxy

Ian Barish

To see what has changed since the last report, view the diff here.

Problem Statement

Background and statement of the theoretical or practical problem, issue or opportunity motivating the research.

A maintainer on a software team is able to publish a package to a public package registry such as PyPI or npm using their credentials and with no other approvals. Nothing requires a second person to agree that this exact artifact should ship, even if multiple developers have equal authority over the project. That single point of authority is the root of a recurring class of supply-chain compromises [1] that are front and center in cybersecurity news today. When one maintainer's account is compromised, whether it be through credential theft [2, 3], a poisoned CI pipeline [4], or a rogue contributor [5], the attacker inherits the unilateral power to push malicious code to every user, company, and dependent package downstream. Recent incidents have followed the same attack shape: one account, a single malicious push, with thousands of victims impacted [3, 6].

The problem is not simply solved by increasing layers of security on developer accounts. The defenses that registries have recently added, such as mandatory two-factor authentication [7] and Trusted Publishing [8], harden authentication, making it harder for an attacker to pretend to be a maintainer. None of them distribute the decision to publish itself. These two layers are orthogonal, authentication hardening does nothing if a threat actor can successfully gain the trust of maintainers, as the XZ Utils backdoor [9] showed. What is missing is authorization granularity, ensuring that several distinct people consent to specific code being shipped before it is sent downstream. The same gap extends beyond software publishing, no general-purpose web authorization solution requires multiple distinct people to cooperate before granting access to a protected resource.

Solution Statement

This research will (the overall aim of the research) in order to (how the research will address the problem) with specific work product/deliverables described.

Enterprise-grade cryptocurrency wallets have long used multi-signature (multi-sig) schemes [10], in which two or more parties must sign a transaction before it leaves a secure wallet. This ensures that the group of individuals that have a vested interest in the wallet authorize anything that happens to it. My research adapts that principle to software publishing, introducing multi-party authorization. I will create a general-purpose multi-party authorization proxy that, for its headline use case, prevents any one individual from publishing

a package to PyPI and instead requires a quorum of reviewers to approve a release before it is pushed downstream. The application is hosted as a web server, and when a developer attempts to publish a package to PyPI, the proxy will:

1. Intercept the request while saving it for publishing
2. Notify other maintainers that a package is attempting to be published
3. Gather enough approvals from the maintainers until a quorum is reached
4. Then forward the publish request on to the PyPI repository

The project will be a proof-of-concept, with the project's broader aim to demonstrate this gap concretely and advocate for multi-party authorization to become a platform-native capability of the registries themselves. The deliverable is a complete, deployable proxy that enforces this quorum-based approval across the full publish lifecycle. The complete scope, user stories, and design decisions are specified in the MVP plan and Product Requirements Document.

Completed Tasks (Last 2 Weeks)

This Progress Report, I changed gears from specification to implementation and built out the MVP of the system using AI assisted development. Working from the specifications that I wrote last PR, I used issues to document functionality, new features, and bugs and implemented it with test-driven development anchored to the specifications across 3 phases of the MVP plan. These phases were organized as vertical slices, backed by executable tests, which made it easy for the AI to implement. I then reviewed each implementation in its own pull request to ensure the code aligned to the spec. This cycle's effort represents roughly 12,700 lines of application and test code across 131 commits, with 83 opened issues, 52 closed issues, and 29 reviewed and merged pull requests.

- Decided on the tech stack for implementation of the system, documented in ADR 0011.
- Broke the implementation down into vertical implementation tasks (issues 1-19).
- Built the full package-publishing request interception. An upload is submitted using standard tooling and is intercepted, held, and bound to the artifact using a SHA-256 hash.
- Implemented the proxy account and approver-authentication model.
- Built the approval core, where approvers can cast individually signed votes in an append-only log.
- Created the post-approval executor, which on quorum re-verifies the artifact hash and publishes to the package repository.
- Implemented the notification and audit systems, so every eligible approver is emailed when a request is pending and every lifecycle event is written to a verifiable audit trail.

- To prove the design's generality, the same approval core drives a second post-approval outcome (a service grant to a traditional web proxy forward-auth).
- Added admin and user self-service portals, declarative provisioning, and dockerized deployment.
- Personally reviewed and merged all 29 pull requests for the cycle, keeping the AI-generated implementation anchored to the specifications.
- Planned the project's evaluation as a four-part framework.

Tasks for the Next Project Report

What will you do over the next two weeks for your project?

1. Finish the threat model into a complete and auditable list. Categorize each threat into executably demonstrated, argued by design, operator-enforced, or an accepted limitation. Align to an industry framework.
2. Build the evaluation suite from that threat model and the project's user stories, collecting and building upon the already-created tests.
3. Run the suite and gather results.
4. Draft the final report outline.

Questions I Have or Issues I'm Running Into

Are there unanticipated obstacles, is something taking longer than you thought it might, or is there an area where the Instructional Team might be able to provide some direction or clarification?

For this cycle, the biggest open question I have is a methodological one concerning the security portion of my evaluation. The headline security result will be a count of how many threats in my threat model are demonstrated by an executable test, argued by the design of the system, enforced by the operator of the system, or accepted as a limitation, with the number of threats in each bucket as the quantitative security score. However, this number is only as trustworthy as the threat list itself, which I, working with AI, authored. This self-enumerated model could risk introducing blind spots that I would never think to test or attack. To mitigate this, I am thinking of grounding the threat model in the MITRE ATT&CK taxonomy [11] to map the threats to and pull inspiration from as I am already familiar with it. However, there are also narrower frameworks [12, 13] that I could pull from that may be more specific to the supply-chain attack use case. I would welcome feedback on whether MITRE ATT&CK is the right anchor for this threat model and if this addresses the "self-grading" issue.

Methodology Paragraph Summary

Description of the methodology you are proposing to get from the starting point of your project to the final deliverable. Projects should employ some systematic process to get to your solution and a clearly identified way to evaluate it to ensure it solves the problem statement identified.

The project will follow an iterative and research-driven development process. I will start by conducting background research on multi-signature authentication cryptography primitives [14, 15, 16, 17, 18, 19, 20, 21], existing authentication systems [22], and any other relevant security design patterns. That research will then inform application specifications for the proxy and approval flow. I will then build a minimum viable product that implements the core flow of intercepting a request, collecting approvals until a quorum is reached, and forwarding the request to the target endpoint. After reviewing the MVP, I will identify any missing requirements, limitations, and security concerns, then return to do additional research and refinement of the specification before implementing the full system. I will then develop an evaluation system that ensures my project solves the problems it was set out to address. This evaluation system is detailed further in the Evaluation section below.

Timeline

Enter one task per row. Include only project tasks (not peer feedback or progress report submissions). Tasks must be actionable. Update the Status column each week (Completed / In Progress / Cancelled / etc.).

Week	Description of Task	Status
W1	Brainstorm ideas for the project	Complete
W1	Create Project Proposal	Complete
W2	Conduct broad project research	Complete
W2	Refine Problem/Solution Statement and Methodology	Complete
W2	Define Project Timeline	Complete
W2	Conduct research on multi-auth crypto primitives	Complete
W3	Develop authentication scheme	Complete
W3	Define high-level architecture and data flow	Complete
W4	Write detailed application specifications	Complete
W4	Create harness for AI assisted programming	Complete
W5	Decide on technology stack	Complete
W5	Develop minimum viable product	Complete
W6	Review MVP to identify missing requirements, limitations, and security concerns	Complete
W6	Develop MVP evaluation framework based on user stories, threat model, and comparisons to other solutions	In Progress
W7	Develop robust threat model tied to an industry framework	In Progress
W7	Build Evaluation suite based on threat model and user stories	Not Started
W8	Finish Evaluation suite and gather results	Not Started
W8	Create report outline	Not Started
W9	Write comparative evaluation arguments	Not Started
W9	Begin writing report	Not Started
W10	Add finishing touches to project implementation	Not Started
W10	Continue writing report	Not Started
W11	Finalize report	Not Started

Evaluation

Include any evaluation plans and/or results by Progress Report 3. This may expand as you finalize the report.

My evaluation will rely on three claims, ordered by how much is my own work and research contribution. It is specified in full in the project's evaluation plan. The first claim is that the system works, the complete package-publishing workflow runs end-to-end and is shown by both executable tests and a runnable demo. Second, that it resists the attacks that it claims to. Each threat in my threat model becomes either an executably demonstrated test, argued by the design of the system, enforced by the operator of the system, or accepted as a limitation, with the number of threats in each bucket as the quantitative security score. Third and finally, that this project fills a gap that no other existing control closes by creating

a cited comparative matrix that establishes what threats each existing control does and doesn't defend compared to my solution.

Two axes are excluded with justifications. First, performance, as my solution relies on human reaction and approval times, which would trump any optimizations I could make to the program. Second, formal human-subject usability, as the system is a proof-of-concept meant to advocate for the functionality to be implemented in more industry solutions.

Report Outline

Include an outline of your final report by Progress Report 4. This may expand as you finalize the report.

References

[1] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," https://doi.org/10.1007/978-3-030-52683-2_2, 2020, DIMVA 2020. Accessed: 2026-06-25.

[2] Python Software Foundation / PyPI Security, "Account takeover and malicious replacement of ctx project," https://python-security.readthedocs.io/pypi-vuln/index-2022-05-24-ctx-domain-takeover.html, 2022, domain-takeover account compromise; disclosed 2022-05-24. Accessed: 2026-06-25.

[3] Cybersecurity and Infrastructure Security Agency, "Widespread supply chain compromise impacting the npm ecosystem," https://www.cisa.gov/news-events/alerts/2025/09/23/widespread-supply-chain-compromise-impacting-npm-ecosystem, 2025, shai-Hulud worm. Accessed: 2026-06-25.

[4] ——, "Emergency directive 21-01: Mitigate SolarWinds Orion code compromise," https://www.cisa.gov/news-events/directives/ed-21-01-mitigate-solarwinds-orion-code-compromise-closed, 2020, SUNBURST build-system compromise; issued 2020-12-13. Accessed: 2026-06-25.

[5] event-stream contributors, ""i don't know what to say." (event-stream / flatmap-stream malicious dependency)," https://github.com/dominictarr/event-stream/issues/116, 2018, GitHub issue documenting the npm compromise. Accessed: 2026-06-25.

[6] Palo Alto Networks Unit 42, ""shai-hulud" worm compromises npm ecosystem in supply chain attack," https://unit42.paloaltonetworks.com/npm-supply-chain-attack/, 2025, accessed: 2026-06-25.

[7] M. Borins, "Top-100 npm package maintainers now require 2FA, and additional security-focused improvements to npm," https://github.blog/security/supply-chain-security/top-100-npm-package-maintainers-require-2fa-additional-security/, 2022, accessed: 2026-06-25.

[8] Python Packaging Authority, "Publishing to PyPI with a trusted publisher," https://docs.pypi.org/trusted-publishers/, 2023, accessed: 2026-06-25.

[9] National Vulnerability Database, "CVE-2024-3094: Malicious code in xz / liblzma (5.6.0–5.6.1)," https://nvd.nist.gov/vuln/detail/CVE-2024-3094, 2024, cVSS 3.1 base score 10.0 (Critical). Accessed: 2026-06-25.

[10] G. Andresen, "BIP 11: M-of-N standard transactions," https://github.com/bitcoin/bips/blob/master/bip-0011.mediawiki, 2011, introduced M-of-N multisignature transactions to Bitcoin. Accessed: 2026-06-25.

[11] MITRE Corporation, "MITRE ATT&CK: A knowledge base of adversary tactics and techniques," https://attack.mitre.org/, globally-accessible knowledge base of adversary tactics and techniques based on real-world observations. Accessed: 2026-06-25.

[12] CNCF TAG Security, "Catalog of supply chain compromises," https://tag-security.cncf.io/community/catalog/compromises/, community catalog of real-world software supply-chain attacks. Accessed: 2026-06-25.

[13] OpenSSF, "SLSA: Supply-chain levels for software artifacts," https://slsa.dev/, accessed: 2026-06-25.

[14] A. Shamir, "How to share a secret," https://doi.org/10.1145/359168.359176, 1979, commun. ACM 22(11):612–613. Accessed: 2026-06-25.

[15] C. Komlo and I. Goldberg, "FROST: Flexible round-optimized Schnorr threshold signatures," https://doi.org/10.1007/978-3-030-81652-0_2, 2020, SAC 2020; IACR ePrint 2020/852. Accessed: 2026-06-25.

[16] J. Nick, T. Ruffing, and Y. Seurin, "MuSig2: Simple two-round Schnorr multisignatures," https://doi.org/10.1007/978-3-030-84242-0_8, 2021, CRYPTO 2021; IACR ePrint 2020/1261. Accessed: 2026-06-25.

[17] R. Gennaro and S. Goldfeder, "Fast multiparty threshold ECDSA with fast trustless setup," https://doi.org/10.1145/3243734.3243859, 2018, ACM CCS 2018 (GG18). Accessed: 2026-06-25.

[18] ——, "One round threshold ECDSA with identifiable abort," https://eprint.iacr.org/2020/540, 2020, IACR ePrint 2020/540 (GG20). Accessed: 2026-06-25.

[19] J. Doerner, Y. Kondi, E. Lee, and A. Shelat, "Secure two-party threshold ECDSA from ECDSA assumptions," https://doi.org/10.1109/SP.2018.00036, 2018, IEEE S&P 2018 (DKLS18). Accessed: 2026-06-25.

[20] ——, "Threshold ECDSA from ECDSA assumptions: The multiparty case," https://doi.org/10.1109/SP.2019.00024, 2019, IEEE S&P 2019 (DKLS19). Accessed: 2026-06-25.

[21] E. Boyle, R. Cohen, D. Data, and P. Hubaček, "Must the communication graph of MPC protocols be an expander?" https://arxiv.org/abs/2305.11428, 2018, CRYPTO 2018; J. Cryptology 36(3), 2023. Accessed: 2026-06-25.

[22] Authelia, "Authelia: Free open-source IAM solution," https://www.authelia.com/, forward-auth reverse-proxy reference architecture. Accessed: 2026-06-25.

Appendix

If there are notes, sources, or figures you want to reference, include them here. Draft work product should begin by Progress Report 3.

AI Usage Disclosure

For transparency, I wanted to include an AI Usage Disclosure in my progress reports encompassing how I've been using AI in my practicum. While artificial intelligence (LLMs) is used throughout this project, it is always under my direction and review. At the start of the project, AI helped me formalize my idea, research, and learn the foundational concepts that the project requires. My primary use of AI has been through iterative, adversarial dialog ("grilling") in which I stress-test my ideas or a design against the AI. I follow Matt Pocock's engineering skills for much of my software development process. Once ideas are settled, I use AI to plan the implementation, such as what vertical slices exist and how each should be built. It then writes the code using test-driven development and checks against the specifications we developed while grilling. I always review every AI-generated specification and implementation against my own understanding of what the project is and must be. The AI submits its programming work as pull requests, and no pull request is merged without my manual review.

For written reports, I use AI to help research, surface credible sources to cite, sharpen my ideas, and produce a rough draft. The meaning and thought behind the words are always my own, and I type every word in the body of these reports to ensure nothing is written without my full understanding, though I may borrow language and sentence structure from the AI-generated draft. I also often have AI review my writing against what we've been grilling on, the project specifications, or the code itself to ensure what I am writing is accurate and my understanding of the project is correct. I believe that my AI use for this project is rigorous; I learn a lot while using it, and it not only helps me be more efficient, but also improves my end product.

Draft Work Product

The draft work product for this project is the working MVP and the design corpus behind it, both in the project repository:

- The MVP implementation built for PR3.
- The application specifications and Architecture Decision Records in the docs/ directory.
- The Product Requirements Document.
- The threat model.
- The evaluation plan that this report summarizes in the Evaluation section.